

Unit 3 Designing Real World Application with SOLID Principles

As requested, Factory Method pattern was used for this exercise.

A method called `create_car()` was created in the parent class `CarFactory` to create `Car` objects (`Sedan`, `SUV`, `Hatchback`), but let subclasses (`SedanFactory`, `SuvFactory`, `HatchbackFactory`) decide which kind of object to create.

The abstractmethod was imported, and a class called `Car` was created with a `drive()` method. Then, the `pass` keyword was used so the method would be syntactically valid without implementing any behaviour yet.

Then, a concrete classes `Sedan`, `SUV`, `Hatchback` were implemented using the same `drive()` method.

Also, an abstractfactory class called `Carfactory` using the abstractmethod was designed with a `create_car()` method returning `Car`. The `pass` keyword was used again.

Then, concrete factories (`SedanFactory`, `SuvFactory`, `HatchbackFactory`) were implemented, using the method `create_car()` and returning its corresponding car type.

Finally, We used a for loop, that iterates through the concrete factories and call `create_car()` and then call `drive()` on the created object.

The SOLID principles were followed. In fact, the main code depends only on the abstract `CarFactory` interface. It doesn't know which concrete factory is being used. This follows the Dependency Inversion Principle. It also follows the Open/Closed principle, since a new car type can be added without modifying any existing class. Polymorphism allows the same `create_car()` call to produce different cars objects depending on the selected factory.

```

from abc import ABC, abstractmethod
# abstract class
class Car(ABC):
    @abstractmethod
    def drive(self):
        pass
#concrete class
class Sedan(Car):
    def drive(self):
        return "2026 Mercedes-Benz E-Class is the best Sedan car"

class Suv(Car):
    def drive(self):
        return "Audi Q5 / Q5 SportBack is the safer SUV"

class Hatchback(Car):
    def drive(self):
        return "Hatchback is the best choice if you need cargo space"

#abstract factory
class CarFactory(ABC):
    @abstractmethod
    def create_car(self) -> Car: # return Car
        pass

#concrete factory
class SedanFactory(CarFactory):
    def create_car(self)-> Car:
        return Sedan()

class SuvFactory(CarFactory):
    def create_car(self)-> Car:
        return Suv()

class HatchbackFactory(CarFactory):
    def create_car(self)-> Car:
        return Hatchback()

for factory in [SedanFactory(),SuvFactory(),HatchbackFactory()]:
    car = factory.create_car()
    print(car.drive())

```